

Gestión de Alumnos

Se proporciona la clase **Main** (ver archivo *Main.java*) que contiene el flujo principal de la aplicación. Esta clase permite seleccionar la implementación de la gestión de alumnos (mediante un array o mediante una colección), y dispone de opciones para insertar alumnos, eliminar, listar, buscar, calcular estadísticas de las notas y determinar la categoría de nota más frecuente.

El alumno debe desarrollar, **a partir del código proporcionado en la clase Main**, el resto de clases siguiendo las siguientes especificaciones.

NO SE PUEDE MODIFICAR LA CLASE MAIN

1. Clase Alumno (RA6)

- **Atributos:**

- DNI (String)
- Nombre (String)
- Edad (int)
- Nota (double)

- **Métodos:**

- Constructores
- Getters and Setters (Métodos accesoros)
- toString
- equals y hashCode
 - En el método equals, para saber si 2 objetos son iguales, es suficiente comparar si tienen el mismo DNI.
 - En el método hashCode, es suficiente con retornar el hashCode del dni.

- **Interfaz Comparable:**

- Es necesario implementar esta interfaz para poder ejecutar en el Main el método *sort()* de la clase *Collection*.
- Se debe ordenar por nombre de forma ascendente

2. Subclases **AlumnoPresencial** y **AlumnoDistancia** (RA7)

- **AlumnoPresencial**

- Además de los atributos heredados de la clase *Alumno*, tiene un nuevo atributo llamado *aula* (String).
- Debes redefinir aquellos métodos que lo necesiten (getters y setters del nuevo atributo y toString)

- **AlumnoDistancia**

- Además de los atributos heredados de la clase *Alumno*, tiene un nuevo atributo llamado *ultimaConexion* (LocalDate), para saber cuándo fue el último momento en el que se conectó a la plataforma de aulavirtual.
- Debes redefinir aquellos métodos que lo necesiten (getters y setters del nuevo atributo y toString)

3. Clase Abstracta **GestionAlumnosAbstract** (RA7)

- **Métodos abstractos:**

- *Alumno* buscarAlumno(*Alumno* alumno);
- void insertar(*Alumno* alumno);
- void eliminar(String dni);
- List<*Alumno*> getAlumnos();
- int getCount();

- **Métodos estáticos (RA4)**

- **Calcular estadísticas de las notas:**

Debe recorrer una lista de alumnos y devolver un *HashMap<String, Integer>* con la cuenta de alumnos por categoría. Las categorías se determinan según la siguiente lógica:

- *Suspenso*: $\text{nota} < 5$
- *Aprobado*: $\text{nota} \geq 5$ y $\text{nota} < 6$

- *Bien*: nota ≥ 6 y nota < 7
- *Notable*: nota ≥ 7 y nota < 9
- *Sobresaliente*: nota ≥ 9

4. Clase **GestionAlumnosArray** (RA4)

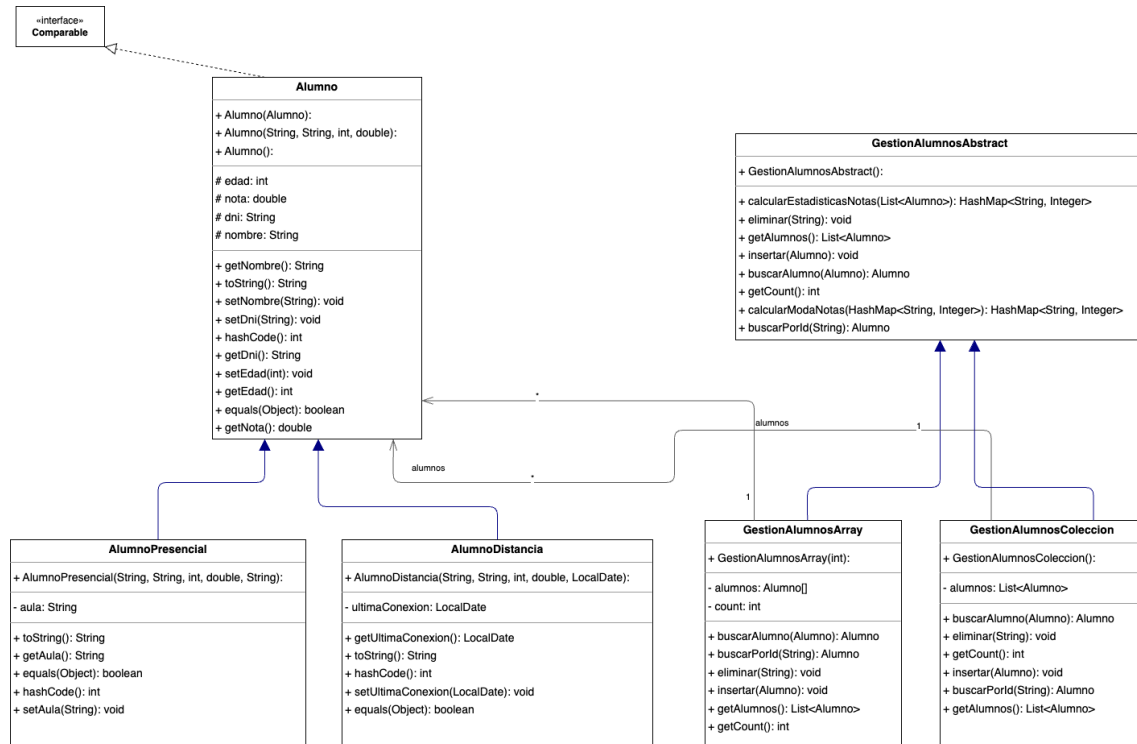
- **Implementación de la gestión** utilizando un **array**:
 - Atributos:
 - La clase debe disponer de un **array** de objetos de tipo **Alumno**
 - Un **contador (int count)** de alumnos, que indique cuantos alumnos hay en el array.
 - Métodos: Implementar correctamente los métodos heredados de **GestionAlumnosAbstract**:
 - **insertar**: Insertar en el array comprobando si hay espacio disponible.
 - **buscarAlumno**: Busca en el array y devuelve el alumno que sea igual al introducido como parámetro.
 - **eliminar**: Eliminar el alumno con el DNI indicado, reajustando el array.
 - **getAlumnos**: Devolver una lista con los alumnos actualmente almacenados.
 - **getCount**: Devolver el número de alumnos insertados.

5. Clase **GestionAlumnosColeccion** (RA4)

- **Implementación de la gestión** utilizando una **colección** (por ejemplo, un ArrayList):
 - La clase debe mantener una colección de objetos **Alumno**.
 - Implementar los métodos heredados de **GestionAlumnosAbstract** de forma que:
 - **insertar**: Añada el alumno a la colección.
 - **buscarAlumno**: Busca en la colección y devuelve el alumno que sea igual al introducido como parámetro.
 - **eliminar**: Elimine el alumno que coincida con el DNI.

- **getAlumnos:** Devuelva una lista (preferiblemente una copia) de los alumnos.
- **getCount:** Devuelva el tamaño de la colección.

6. Diagrama de clases



7. Ejecución del programa.

Opción: 1

--- Menú de Gestión de Alumnos ---

1. Insertar alumnos
2. Mostrar todos los alumnos (ordenados)
3. Eliminar alumno (elimina DNI: 2222222B)
4. Mostrar número de alumnos creados
5. Buscar alumno por DNI (buscando DNI: 3333333C)
6. Mostrar estadísticas de las notas
7. Obtener la categoría de nota mas frecuente
0. Salir

Opción: 1

Alumnos hardcoded insertados correctamente.

--- Menú de Gestión de Alumnos ---

1. Insertar alumnos
2. Mostrar todos los alumnos (ordenados)
3. Eliminar alumno (elimina DNI: 22222222B)
4. Mostrar número de alumnos creados
5. Buscar alumno por DNI (buscando DNI: 33333333C)
6. Mostrar estadísticas de las notas
7. Obtener la categoría de nota mas frecuente
0. Salir

Opción: 2

Listado de alumnos (ordenados por nombre):

Alumno{DNI='11111111A', Nombre='Alice', Edad=20, Nota=7.5}, Aula='Aula 101'
Alumno{DNI='22222222B', Nombre='Bob', Edad=22, Nota=4.5}, UltimaConexion=2023-03-15
Alumno{DNI='33333333C', Nombre='Charlie', Edad=19, Nota=9.0}, Aula='Aula 102'
Alumno{DNI='44444444D', Nombre='Diana', Edad=21, Nota=6.8}, UltimaConexion=2023-03-16
Alumno{DNI='55555555E', Nombre='Eva', Edad=20, Nota=8.5}, Aula='Aula 103'
Alumno{DNI='66666666F', Nombre='Frank', Edad=23, Nota=4.0}, UltimaConexion=2023-03-17
Alumno{DNI='77777777G', Nombre='Grace', Edad=21, Nota=6.5}, Aula='Aula 104'
Alumno{DNI='88888888H', Nombre='Héctor', Edad=19, Nota=7.0}, UltimaConexion=2023-03-18
Alumno{DNI='99999999I', Nombre='Irene', Edad=22, Nota=8.0}, Aula='Aula 105'
Alumno{DNI='10101010J', Nombre='James', Edad=20, Nota=9.5}, UltimaConexion=2023-03-19

--- Menú de Gestión de Alumnos ---

1. Insertar alumnos
2. Mostrar todos los alumnos (ordenados)
3. Eliminar alumno (elimina DNI: 22222222B)
4. Mostrar número de alumnos creados
5. Buscar alumno por DNI (buscando DNI: 33333333C)
6. Mostrar estadísticas de las notas
7. Obtener la categoría de nota mas frecuente
0. Salir

Opción: 3

SE ELIMINA, PERO NO MUESTRA NADA
POR PANTALLA

--- Menú de Gestión de Alumnos ---

1. Insertar alumnos
2. Mostrar todos los alumnos (ordenados)
3. Eliminar alumno (elimina DNI: 22222222B)
4. Mostrar número de alumnos creados
5. Buscar alumno por DNI (buscando DNI: 33333333C)
6. Mostrar estadísticas de las notas
7. Obtener la categoría de nota mas frecuente
0. Salir

Opción: 4

Número de alumnos creados: 9

--- Menú de Gestión de Alumnos ---

1. Insertar alumnos
2. Mostrar todos los alumnos (ordenados)
3. Eliminar alumno (elimina DNI: 22222222B)
4. Mostrar número de alumnos creados
5. Buscar alumno por DNI (buscando DNI: 33333333C)
6. Mostrar estadísticas de las notas
7. Obtener la categoría de nota mas frecuente
0. Salir

Opción: 5

Alumno encontrado: Alumno{DNI='33333333C', Nombre='Charlie', Edad=19, Nota=9.0}, Aula='Aula 102'

--- Menú de Gestión de Alumnos ---

1. Insertar alumnos
2. Mostrar todos los alumnos (ordenados)
3. Eliminar alumno (elimina DNI: 22222222B)
4. Mostrar número de alumnos creados
5. Buscar alumno por DNI (buscando DNI: 33333333C)
6. Mostrar estadísticas de las notas
7. Obtener la categoría de nota mas frecuente
0. Salir

Opción: 6

Estadísticas de notas:

Suspenso: 1
Sobresaliente: 2
Notable: 4
Aprobado: 0
Bien: 2

--- Menú de Gestión de Alumnos ---

1. Insertar alumnos
2. Mostrar todos los alumnos (ordenados)
3. Eliminar alumno (elimina DNI: 22222222B)
4. Mostrar número de alumnos creados
5. Buscar alumno por DNI (buscando DNI: 33333333C)
6. Mostrar estadísticas de las notas
7. Obtener la categoría de nota mas frecuente
0. Salir

Opción: 7

La moda de las categorías de las notas es: {Notable=4}

Criterios de Evaluación

RA's	Qué se valora
RA4	2,5pt - Aplica correctamente los modificadores de acceso 3pt - Crea constructores apropiados 1,5pt - Crea métodos accesorios (getters y setters) correctamente 1,5pt - Crea método toString() correctamente 1,5pt - Crea los métodos equals() y hashCode() correctamente
RA6	3pt - Implementa correctamente los métodos que manejan Arrays 3pt - Implementa correctamente los métodos que manejan la interfaz List 4pt - Implementa correctamente los métodos estáticos que usan la interfaz Map
RA7	5pt - Implementa correctamente la herencia de las clases hijas 1pt - Implementa correctamente las relaciones de composición 2pt Implementa correctamente la clase abstracta y redefine sus métodos. 2pt - Implementa correctamente la interfaz en la clase Alumno

Entrega

- Se debe entregar el proyecto con todas las clases implementadas.
- El código fuente debe compilar y ejecutarse sin errores.
- El alumno debe incluir un fichero README breve que explique cualquier decisión de diseño relevante.